OOPs Practice Questions

🟢 Easy Level (Q1 - Q4)

Question 1 — Understanding `this` in an Object

```
const user= {
  name:"Ritik",
  greet() {
console.log(this.name);
  }
};
```

What will be printed when:

```
user.greet();
```

Scenario

A profile page wants to display the logged-in user's name.

What is this question asking?

You need to identify what `this` refers to.

Remember:

> "Dot ke left mein jo object hai, wahi `this` hai."

Here:

```
user.greet()
```

means:

```
this===user
```

Concepts Tested

- this keyword
- Method calls

Question 2 — Default Binding

```
function show() {
console.log(this);
}

show();
```

What will be printed?

Also explain the difference between:

- Browser
- Node.js
- Strict Mode

Scenario

You copied code from a browser project into Node.js and got different results.

What is this question asking?

Understand what happens when a regular function is called without an object.

Concepts Tested

- this
- Default Binding
- Strict Mode

Question 3 — call()

```
function introduce() {
console.log(this.name);
}

const person= {
  name:"Ritik"
};
```

Print:

```
Ritik
```

using `call()`.

Scenario

You want to borrow a function for another object.

What is this question asking?

Manually decide what `this` should be.

Concepts Tested

- call()

Question 4 — apply()

```
function introduce(city,country) {
console.log(`${this.name} from${city}`);
}

const person= {
  name:"Ritik"
};
```

Call the function using `apply()`.

Scenario

Arguments are coming inside an array from an API.

What is this question asking?

Use apply instead of call.

Concepts Tested

- apply()
- Explicit Binding

🟡 Moderate Level (Q5 - Q7)

Question 5 — Fix Lost `this`

```
const user= {
  name:"Ritik",
  greet() {
console.log(this.name);
  }
};
```

```
const fn=user.greet;

fn();
```

The code prints:

```
undefined
```

Fix it using `bind()`.

Scenario

A method was passed as a callback and lost its object reference.

What is this question asking?

Understand why `this` gets lost and how bind permanently fixes it.

Concepts Tested

- bind()
- this
- Callback Problems

Question 6 — Create an Inheritance Chain

Create:

```
const animal= {
  eats:true
};
```

Then create:

```
const dog
```

using `Object.create()`.

The dog object should inherit:

```
eats
```

from animal.

Scenario

You are building a game where many animals share common behaviors.

What is this question asking?

Create an object whose prototype is another object.

Concepts Tested

- Object.create()
- Prototype Chain

Question 7 — Prototype Method

Create:

```
function Person(name) {
this.name=name;
}
```

Add:

```
greet()
```

using:

```
Person.prototype
```

so every person object can use it.

Scenario

Thousands of users need the same method.

You don't want a separate copy for every user.

What is this question asking?

Store methods inside the prototype instead of the object itself.

Concepts Tested

- Constructor Functions
- Prototype
- Memory Optimization

🔴 Hard Level (Q8 - Q10)

Question 8 — Student Class System

Create a class:

```
class Student
```

Properties:

```
name
marks
```

Methods:

```
getGrade()
```

Rules:

```
90+ =>A
75+ =>B
60+ =>C
Otherwise =>F
```

Scenario

School Report Card System.

What is this question asking?

Create a real-world class that stores data and behavior together.

Concepts Tested

- Classes
- Constructor
- Instance Methods

Question 9 — Employee Inheritance

Create:

```
class Employee
```

Properties:

```
name
salary
```

Method:

```
work()
```

Then create:

```
class Developer
```

that extends Employee.

Add:

```
code()
```

method.

Scenario

Company Management Software.

What is this question asking?

Build a parent-child relationship.

Expected:

```
dev.work();
dev.code();
```

both should work.

Concepts Tested

- Classes
- extends
- super()
- Inheritance

Question 10 — Bank Account (Interview-Level)

Create:

```
class BankAccount
```

Requirements:

Private Field

```
#balance
```

Methods

```
deposit(amount)
withdraw(amount)
getBalance()
```

Rules:

- Cannot withdraw more than available balance.
- Balance should never be directly accessible.

Example:

```
const acc = newBankAccount();

acc.deposit(1000);
acc.withdraw(300);

console.log(acc.getBalance());
```

Output:

```
700
```

Scenario

Banking Application.

What is this question asking?

This question combines:

1. Classes
2. Private Fields
3. Encapsulation
4. Business Logic

You must protect data and expose only controlled operations.

Concepts Tested

- Classes
- Private Fields (`#`)
- Methods
- Encapsulation
- Real-world Design

🎯 Bonus Challenge (Very Hard)

Create a complete:

```
Library Management System
```

using Classes.

Class: Book

Properties:

```
title
author
borrowed
```

Class: Library

Methods:

```
addBook()
borrowBook()
returnBook()
showAvailableBooks()
```